

FRED TALKS

4th August 2026

CONTENTS

LLM Evaluation: Practical Tips at Booking.com

GEORGE CHOULIARAS · 2734 WORDS

Getting Real About Distributed System Reliability

BOREDANDROID · 2792 WORDS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 1776 WORDS

Maze

XKCD.COM · 23 WORDS

LLM Evaluation: Practical Tips at Booking.com

GEORGE CHOULIARAS · 13 SEP 2025 · [SOURCE](#)

Lessons learned from 1 year of Judge-LLM Development

By

Zeno Belligoli

,

Antonio Castelli

,

George Chouliaras



The increasing adoption of Large Language Models (LLMs) across various industries has made evaluating LLM-powered applications (also called Generative AI applications) a critical necessity.

LLMs are pre-trained on a vast amount of text and can therefore be used to perform a wide range of tasks based on a natural language input called **prompt**. These tasks range from extracting information, answering complex questions, summarizing documents, generating creative content, translating languages, writing code etc.

However, unlike traditional Machine Learning (ML) models, the nature of LLMs brings inherent challenges that must be carefully considered:

- LLMs can **hallucinate**, i.e. can generate outputs which are factually incorrect or nonsensical while presenting them with high confidence
- LLMs can fail to **follow the instructions** specified in the prompt no matter how detailed they sound to a human reader
- Usage of LLMs has a **non-negligible cost** whether the model is open-source and served in-house or closed-source and accessed via a paid API.

To maximize the potential of Generative AI (GenAI) applications and mitigate the risks associated with them, we built a framework capable of thoroughly evaluating the performance of an LLM on a specific task in a nearly automated way. This framework is based on the concept of **LLM-as-judge**, i.e. on the usage of a more powerful LLM to evaluate the “target” LLM.

The LLM-as-a-judge framework

The main difference between the evaluation of traditional ML models and LLMs is that in most of the cases we are dealing with generative tasks for which either no single ground truth exists or is hard / expensive to obtain. When we ask an LLM to write a description of a property, or to summarize a long text, we don’t really have an unambiguous reference that we can use for comparison. Moreover we can’t get this at scale for different topics or subjects.

Therefore the measurement of text attributes like clarity, readability, toxicity or factual accuracy becomes more challenging. It would be possible, in theory, to employ human experts to review every generation and provide an estimation of the metric under study. However, this process would be time consuming and expensive to be **practically infeasible**.

The LLM-as-judge approach, requires human involvement **only once** (unless the production distribution changes), to carefully annotate the so-called **golden dataset** which must be large enough to be representative of the data distribution in production (the best practices for golden dataset creation are described in the next section).

Once we have the golden dataset with labels, we can prompt (or fine-tune) an LLM to replicate human judgement as much as possible. Once this is achieved, (e.g. when the agreement between the judge LLM and the human annotation has an accuracy above a certain threshold) the judge LLM can be employed to score the predictions of a second LLM (*target LLM*) on other datasets.

This allows the continuous monitoring of the performance of a GenAI application in production in a scalable way with minimal human involvement.

Since the judge-LLM solves a classification task (e.g. is the text clear or not? is the text factually accurate or not?) getting golden labels is much easier. Hence, we can evaluate its performance using standard metrics like accuracy, f1-score, precision or recall, to monitor the performance of the target LLM. The process of judge-LLM development and usage is depicted in Figure 1.

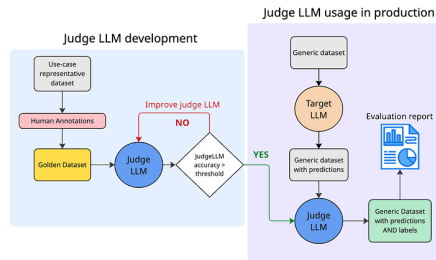


Figure 1: Judge-LLM development and deployment cycle. The left box (light-blue) shows the judge-LLM development phase. This phase always starts with a golden dataset with labels provided by a fully trusted source (e.g. human experts). A base LLM model is prompted or fine-tuned to imitate human label scoring. When the judge-LLM reaches a certain accuracy with respect to the golden labels, it can be deployed in production (right purple box) to score any other similar dataset, for example to evaluate the output of a target LLM. An evaluation report can be produced using the predictions of the target LLM and the label provided by the Judge LLM.

Golden Dataset Creation

The main aim of a golden dataset is to **accurately assess the ability** of a judge-LLM, to evaluate a production LLM in a particular evaluation metric. Due to this, a **high quality** golden dataset should meet the following criteria:

- It should represent the *production distribution*, since we want the Judge LLM evaluation to be as close to production as possible.
- It should include *golden* labels, e.g. labels with high quality. This can be proxied by high inter-annotator agreement among the annotators of the labels.

Low inter-annotator agreement usually indicates ambiguity in the metric definition and should be resolved by calibration among the annotators and by adding clarifications in the definition until the agreement is acceptable.

Since text attributes often do not have an objective and unique true value, achieving high-quality annotations is not a trivial task. To make fair comparisons between different versions of a GenAI application it is of utmost importance to have a **standardized annotation protocol** which is rigorously followed. Below we provide both a **basic** and an **advanced** annotation protocol for creating the golden dataset.

Basic Annotation Protocol

The basic annotation protocol can be used when only a single annotator is available.

1. Metric Definition

Define the metric definition with the business owner in the most unambiguous way possible.

- Write a **clear** definition of the metric as objective as possible. Prefer **binary** metrics or categorical metrics with a few classes, since LLMs struggle with continuous scores. If you're forced to use a continuous metric, consider binning into a few coarse ordinal values e.g. 'high', 'medium', 'low'.
- Write **annotation instructions**, using the metric definition. These are the instructions a human should follow in order to provide the ground truth label. Describe the task **clearly** and **objectively**. Avoid vague words (such as major, minor, a few). Provide scored examples and include edge cases. Specify what tools the annotators can and cannot use (Search engines, ChatGPT, etc).

2. Pilot Annotation

Collect a few examples (~50) and annotate them according to the aforementioned metric definition. The annotators should carefully follow the instructions provided, without bringing any previous knowledge or bias on the task. The annotators at this stage can be selected from business owners or from other technical people involved. The aim of this step is to ensure that we have a clear metric definition and that the annotations are aligned with it.

Once the annotations are done, it is recommended that they are **reviewed** by a pool of domain experts to assess if they align with the guidelines. If issues or disagreements are found, they should be resolved, the metric definition should be updated and the pilot annotation should be repeated until full consensus is reached.

3. Full Annotation by a single annotator

Once the annotation of the first small sample during the pilot annotation is finalized and reviewed, the examples can be shared to the annotator as guidance in order to annotate a full dataset. The full dataset should consist of **ideally 500- 1000 examples** at least. This is because part of the dataset will be used for tuning the judge-LLM (validation set) and another part as a holdout set in order to evaluate its performance.

4. Quality Review

Get George Chouliaras's stories in your inbox

Join Medium for free to get updates from this writer.

A subject matter expert reviews a **representative** sample (~10%) of the full annotation. If a significant percentage (> 10%) of the labels in the sample are incorrect, annotation should be repeated. The quality thresholds can be adjusted according to the organization's needs and should be communicated and agreed upon with the annotators before the full annotation.

Advanced Annotation Protocol

The advanced annotation protocol requires more effort but also guarantees higher quality annotations than the basic one.

1. Metric Definition

Same as in the basic protocol.

2. Pilot Annotation

Same as in the basic protocol.

3. Full Annotation by multiple annotators

Instead of having a single human annotator providing the label for a given example, we have multiple human annotators (3+) providing a label for each example.

Depending on the use case, an aggregation function must be defined to obtain the final label.

The aggregation function should take an array of labels and return a single one. In principle any

aggregation function can be used. If labels can be converted into integers and ordered any function like min, max, or average can be used. If labels have no orders (e.g. geographical location) majority vote can be used.

Examples where the annotators **disagree** can be treated in multiple ways, depending on what is more appropriate for the specific problem.

a) Add a “not sure” class: map to the “not sure” class all examples without 100% consensus

b) Add a weight < 1 : For all examples without 100% consensus select the majority class among the annotated ones (a random one if there is no majority) and add a weight to the example equal to $n_selected_class_votes / n_total_votes$. This approach will penalize the more uncertain examples in the aggregated performance.

c) Discard the ambiguous examples: For some use cases it might make sense to simply remove all the examples where there is no 100% agreement.

4. Quality Review

Same as in the basic protocol.

How to develop a judge-LLM

Once the golden dataset is ready, you can develop the judge-LLM. Note that you can already do this step once the sample dataset from the pilot annotation is ready (with ~50 samples) to create a first version of the judge-LLM and score a few examples to identify any early issues. However for proper evaluation and tuning of the judge-LLM you need a large dataset of 500–1000 examples. In this article we share how to create a prompt-based judge-LLM for simplicity. You can always fine-tune a judge-LLM but this won't be covered here. To develop the judge-LLM you follow the **iterative process** detailed below, which is the standard process for manual prompt engineering.

1. **Split the golden dataset in validation and test sets:** A standard split is 50/50 but the final decision is on the model owner. The validation set is used to compute the prompt performance and to find error patterns. The test set is used to check for evaluating generalization and overfitting.
2. **Choose a strong LLM:** We typically select powerful models such as **GPT-4.1**, or **Claude 4.0 Sonnet** as the backbone for our judge-LLM. These models serve two roles:

a) As a sanity check: Given a high-quality dataset and a well-defined task, a strong LLM should perform well if the prompt is reasonable.

b) To estimate an upper bound on achievable performance. While this is not a strict bound (there are cases where smaller models with better prompts outperform stronger ones) it provides a useful reference point.

3. Write the prompt of the judge-LLM: Use the metric definition and include scoring instructions and output format (can be json/string/else). Few shot examples can also be included but ensure that no examples from the golden dataset are provided to avoid overfitting. We also normally add chain-of-thought prompting (CoT) to promote reasoning and explainability. To build the judge-LLM you can use any framework that supports custom LLM-based metrics, such as DeepEval's G-Eval metric or Arize Phoenix.

4. Evaluate the performance on the validation set: The evaluation metric depends on the type of the task and the class imbalance. For classification tasks we generally recommend macro f1-score while for continuous metric you can use mean squared error.

5. Perform error analysis and update the prompt

We analyze the model's mistakes on the validation set to identify recurring issues or edge cases. Based on these insights, we revise the prompt to better align it with the desired behavior. We then repeat steps 3, 4 and 5. Typically, we are satisfied once we reach a value of the evaluation metric above a certain threshold agreed with the business stakeholders.

6. Evaluate the performance on the test set: This will give the unbiased performance score of the judge-LLM. It is not recommended to change the prompt in order to improve the performance on the test set, as this might end up in overfitting on this dataset.

After prompt engineering is completed with a strong model, we repeat the same process using a **weaker (and more cost-efficient) model**, aiming to match the performance of the stronger version as closely as possible. In practice, we use the **strong-model judge-LLM during AI system development**, where quality is critical, and **the weaker-model version for large-scale monitoring** of system performance in production environments. An example of such monitoring is provided in Figure 2.

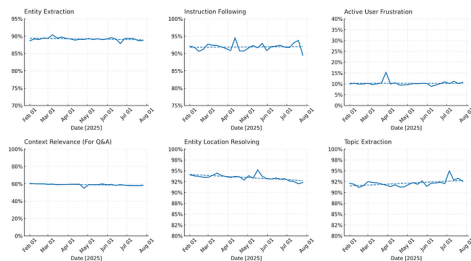


Figure 2: The figure shows an example dashboard used to monitor the quality of an LLM application, based on judge-LLM metrics. Metric values can also be combined to get aggregated scores. In this particular example we show the trends for Entity Extraction accuracy, LLM-instruction-following accuracy, frustration of users, context relevance, LLM-location-resolving accuracy and LLM-topic-conversation-extraction accuracy. From the plot it is immediate to quickly spot issues connected to instruction-following and topic-extraction allowing for prompt mitigation actions. An automated system is set-up to alert the application owners whenever an anomaly is detected for any of the relevant metrics.

Future directions

Pointwise vs. Comparative Judges

The vast majority of our judge-LLMs are **pointwise judges**, which assign an absolute score to each response independently. Pointwise judges are particularly useful because they can serve **dual purposes**:

- Ranking system outputs during development.
- Monitoring performance in production, where only one system's response is typically available.

However, **comparative (pair-wise)** judges (which compare two responses and select the better one) tend to offer **stronger ranking signals**. It is often easier for a model to decide which of two answers is better than to assign calibrated absolute scores. For this reason, comparative judges can be more effective during the AI system development phase. We are actively exploring methods to **develop both pointwise and comparative judges with minimal extra overhead**, ideally reusing most of the dataset and prompt engineering efforts.

Automatic judge-LLM development

Prompt engineering remains a **manual, iterative, and time-intensive process**, typically taking anywhere from **one day to a full week** depending on the complexity of the task. To streamline this, we have developed an **automated prompt engineering pipeline** inspired by [DeepMind's OPRO](#) that, in essence, automates **steps 3–5** of the process. At the end of the process it is always important that the human inspects the final prompt to ensure that it doesn't contain use case specific examples or rules that might undermine generalizability.

Another bottleneck in the judge-LLM development process is that of data annotation. Annotating a good dataset requires substantial effort from multiple annotators and can take from some days to weeks, depending on the complexity of the task. In this context, one can leverage the ability of LLMs to generate realistic text to create **synthetic data**. This is particularly helpful and achievable for classification tasks, whereby the LLM only needs to generate the input X, conditioned on a certain class Y. Substantial challenges in this area lie in the generation of diverse and realistic text and conversations and in the quality evaluation of the synthetic data, that is, how do we know that our synthetic data is realistic, diverse and of high-quality? We are actively working on this area and are looking forward to sharing more with you in a future blog post.

Evaluating LLM agents

We did not cover LLM-based **agent evaluation** in this article as it is a topic on its own. This is because evaluating agents often requires assessing long-term goal completion, reasoning over multi-step interactions, handling complex tool use and effective use of planning and memory. These are challenges that go beyond the evaluation of isolated model outputs and typically involve different methodologies, benchmarks, and infrastructure. We will cover this topic in a separate blog post.

Key takeaways

Evaluating generative tasks is inherently complex due to the absence of clear ground truth data. The LLM-as-a-judge framework offers an efficient method to scale this evaluation process in an automated way. A high-quality judge-LLM is built upon a “golden dataset” with reliable labels, which necessitates a rigorous annotation protocol; we have outlined both a basic and an advanced version to achieve this. This judge-LLM can then be optimized through manual, iterative prompt engineering or via automated methods. Future directions in this field include creating golden datasets with synthetic data and developing evaluation methods for LLM-based agents.

Getting Real About Distributed System Reliability

BOREDANDROID · 01 MAR 2026 · [SOURCE](#)

There is a lot of hype around distributed data systems, some of it justified. It's true that the internet has centralized a lot of computation onto services like Google, Facebook, Twitter, LinkedIn (my own employer), and other large web sites. It's true that this centralization puts a lot of pressure on system scalability for these companies. It's true that incremental and horizontal scalability is a *deep* feature that requires redesign from the ground up and can't be added incrementally to existing products. It's true that, if properly designed, these systems can be run with no *planned* downtime or maintenance intervals in a way that traditional storage systems make harder. It's also true that software that is explicitly designed to deal with machine failures is a very different thing from traditional infrastructure. All of these properties are critical to large web companies, and are what drove the adoption of horizontally scalable systems like [Hadoop](#), [Cassandra](#), [Voldemort](#), etc. I was the original author of Voldemort and have worked on distributed infrastructure for the last four years or so. So in-so-far as there is a "big data" debate, I am firmly in the "pro-" camp. But one thing you often hear is that this kind of software is more reliable than the traditional alternatives it replaces, and this just isn't true. It is time people talked honestly about this.

You hear this assumption of reliability everywhere. Now that scalable data infrastructure has a marketing presence, it has really gotten bad. Hadoop or Cassandra or what-have-you can tolerate machine failures then they must be unbreakable right? Wrong.

Where does this come from? Distributed systems need to partition data or state up over lots of machines to scale. Adding machines increases the probability that some machine will fail, and to address this these systems typically have some kind of replicas or other redundancy to tolerate failures. The core argument that gets used for these systems is that if a single machine has probability P of failure, and if the software can replicate data N times to survive $N-1$ failures, and if the machines fail *independently*, then the probability of losing a particular piece of data must be P^N . So for any desired reliability R and any single-node failure probability P you can pick some replication N so that $P^N < R$. This argument is the core motivation behind most variations on replication and fault tolerance in distributed systems. It is true that without this property the system would be hopelessly unreliable as it grew. But this leads people to believe that distributed software is somehow innately reliable, which unfortunately is utter hogwash.

Where is the flaw in the reasoning? Is it the dreaded Hadoop single points of failure? No, it is far more fundamental than that: the problem is the assumption that failures are independent. Surely no belief could possibly be more counter to our own experience or just common sense than believing that there is no correlation between failures of machines in a cluster. You take a bunch of pieces of identical hardware, run them on the same network gear and power systems, have the same people run and manage and configure them, and run the same (buggy) software on all of them. It would be incredibly unlikely that the failures on these machines would be independent of one another in the probabilistic sense that motivates a lot of distributed infrastructure. If you see a bug on one machine, the same bug is on all the machines. When you push bad config, it is usually game over no matter how many machines you push it to.

P^N is an upper bound on reliability but one that you could never, never approach in practice. For example Google has a fantastic paper that gives empirical numbers on system failures in Bigtable and GFS and reports empirical data on groups of failures that show rates several orders of magnitude higher than the independence assumption would predict. This is what one of the best system and operations teams in the world can get: your numbers may be far worse.

The actual reliability of your system depends largely on how bug free it is, how good you are at monitoring it, and how well you have protected against the myriad issues and problems it has. This isn't any different from traditional systems, except that the new software is far less mature. I don't mean this disparagingly, I work in this area, it is just a fact. Maturity comes with time and usage and effort. This software hasn't been around for as long as MySQL or Oracle, and worse, the expertise to run it reliably is much less common. MySQL and Oracle administrators are plentiful, but folks experience with, say, serious production Zookeeper operations knowledge are much more rare.

Kernel filesystem engineers say it takes about a decade for a new filesystem to go from concept to maturity. I am not sure these systems will be mature much faster—they are not easier systems to build and the fundamental design space is much less well explored. This doesn't mean they won't be *useful* sooner, especially in domains where they solve a pressing need and are approached with an appropriate amount of caution, but they are not yet by any means a mature technology.

Part of the difficulty is that distributed system software is actually quite complicated in comparison to single-server code. Code that deals with failure cases and is “cluster aware” is extremely tricky to get right. The root of the problem is that dealing with failures effectively explodes the possible state space that needs testing and validation. For example it doesn't even make sense to expect a single-node database to be fast if its disk system suddenly gets really slow (how could it), but a distributed system does need to carry on in the presence of single degraded machine because it has some many machines, one is sure to be degraded. These kind

of “semi-failures” are common and very hard to deal with. Correctly testing these kinds of issues in a realistic setting is brutally hard and the newer generation of software doesn’t have anything like the QA processes its more mature predecessors had. (If you get a chance get someone who has worked at Oracle to describe to you what kind of testing they do to a line of code that goes into their database before it gets shipped to customers). As a result there are a lot of bugs. And of course these bugs are on all the machines, so they absolutely happen together.

Likewise distributed systems typically require more configuration and more complex configuration because they need to be cluster aware, deal with timeouts, etc. This configuration is, of course, shared; and this creates yet another opportunity to bring everything to its knees.

And finally these systems usually want lots of machines. And no matter how good you are, some part of operational difficulty always scales with the number of machines.

Let’s discuss some real issues. We had a bug in [Kafka](#) recently that lead to the server incorrectly interpreting a corrupt request as a corrupt log, and shutting itself down to avoid appending to a corrupt log. Single machine log corruption is the kind of thing that should happen due to a disk error, and bringing down the corrupt node is the right behavior—it shouldn’t happen on all the machines at the same time unless all the disks fail at once. But since this was due to corrupt *requests*, and since we had one client that sent corrupt requests, it was able to sequentially bring down all the servers. Oops. Another example is [this Linux bug](#) which causes the system to crash after ~200 days of uptime. Since machines are commonly restarted sequentially this lead to a situation where a large percentage of machines went hard down one after another. Likewise any memory management problems—either leaks or GC problems—tend to happen everywhere at once or not at all. Some companies do public post-mortums for major failures and these are a real wealth of failures in systems that aren’t supposed to fail. [This paper](#) has an excellent summary of HDFS availability at Yahoo—they note how few of the problems are of the kind that high availability for the namenode would solve. This list could go on, but you get the idea.

I have come around to the view that the real core difficulty of these systems is operations, not architecture or design. Both are important but good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations. This is quite different from the view of unbreakable, self-healing, self-operating systems that I see being pitched by the more enthusiastic NoSQL hypesters. Worse yet, you can’t easily buy good operations in the same way you can buy good software—you might be able to hire good people (if you can find them) but this is more than just people; it is practices, monitoring systems, configuration management, etc.

These difficulties are one of the core barriers to adoption for distributed data infrastructure. LinkedIn and other companies that have a deep interest in doing creative things with data have taken on the burden of building this kind of expertise in-house—we employ committers on Hadoop and other open source projects on both our engineering and operations team, and have done a lot of from-scratch development in this space where there was gaps. This makes it feasible to take full advantage of an admittedly valuable but immature set of technologies, and let's us build products we couldn't otherwise—but this kind of investment only makes sense at a certain size and scale. It may be too high a cost for small startups or companies outside the web space trying to bootstrap this kind of knowledge inside a more traditional IT organization.

This is why people should be excited about things like Amazon's DynamoDB. When DynamoDB was released, the company DataStax that supports and leads development on Cassandra released a feature comparison checklist. The checklist was unfair in many ways (as these kinds of vendor comparisons usually are), but the biggest thing missing in the comparison is that *you* don't run DynamoDB, Amazon does. That is a huge, huge difference. Amazon is good at this stuff, and has shown that they can (usually) support massively multi-tenant operations with reasonable SLAs, *in practice*.

I really think there is really only one thing to talk about with respect to reliability: continuous hours of successful production operations. That's it. In some ways the most obvious thing, but not typically what you hear when people talk about these systems. I will believe the system can tolerate (some) failures when I see it tolerate those failures; I believe it can run for a year without downtime when I see it run for a year without downtime. I call this empirical reliability (as opposed to theoretical reliability). And getting good empirical reliability is really, really hard. These systems end up being large hunks of monitoring, tests, and operational procedures with a teeny little distributed system strapped to the back.

You see this showing up in discussions of the CAP theorem all the time. The CAP theorem is a useful thing, but it applies more to system design than system implementations. A design is simple enough that you can maybe prove it provides consistency or tolerates partition failures under some assumptions. This is a useful lens to look at system designs. You can't hope to do this kind of proof with an actual system implementation, the thing you run. The difficulty of building these things means it is really unthinkable that these systems are, in actual reality, either consistent, available, *or* partition tolerant—they certainly all have numerous bugs that will break each of these guarantees. I really like this paper that takes the approach of actually trying to calculate the observed consistency of eventually consistent systems—they seem to do it via simulation rather than measurement, which is unfortunate, but the idea is great.

It isn't that system design is meaningless, it is worth discussing the system design as it does act as a kind of limiting factor on certain aspects of reliability and performance as the implementation matures and improves, but don't take it too seriously as guaranteeing *anything*.

So why isn't this kind of empirical measurement more talked about? I don't know. My pet theory is it has to do with the somewhat rigid and deductive mindset of classical computer science. This is inherited from pure math, and conflicts with the attitude in scientific disciplines. It leads to a preference for starting with axioms, and then proving various properties that follow from these axioms. This world view doesn't embrace the kind of empirical measurements you would expect to justify claims about reality (for another example see [this great blog post](#) on programming language productivity claims in programming language research). But that is getting off topic. Suffice it to say, when making predictions about how a system will work in the real world I believe in measurements of reality a lot more than arguments from first-principles.

I think we should insist on a little more rigor and empiricism in this area.

I would love to see claims in academic publication around practicality or reliability justified in the same way we justify performance claims—by doing it. I would be a lot more likely to believe an academic distributed system was practically feasible if it was run continuously under load for a year successfully and if information was reported on failures and outages. Maybe that isn't feasible for an academic project, but few other allegedly scientific academic disciplines can get away with making claims about reality without evidence.

More broadly I would like to see more systems whose *operation* is verifiable. Many systems have the ability to log out information about their state in a way that makes programmatic checking of various invariants and guarantees possible (such as consistency or availability). An example of such an invariant for a messaging system is that all the messages sent to it are received by all the subscribers. We actually measure the truth of this statement in real-time in production for our Kafka data pipeline for all 450 topics and all their subscribers. The number of corner-cases one uncovers with this kind of check, run through a few hundred use cases and a few billion messages per day is truly astounding. I think this is a special case of a broad class of verification that can be done on the running system that goes far far deeper than what is traditionally considered either monitoring or testing. Call it unit testing in production, or self-proving systems, or next generation monitoring, or whatever, but I think this kind of deep verification is something that makes turning the theoretical claims a design makes into measured properties of the real running system.

Likewise if you have a “NoSQL vendor” I think it is reasonable to ask them to provide hard information on customer outages. They don't need to tell you who the customer is, but they should let you know the observed real-life distribution of MTTF and MTTR they are able to

achieve, not just highlight one or two happy cases. Make sure you understand how they measure this, do they have automated test load that runs or just wait for people to complain? This is a reasonable thing for people paying for a service to ask for. To a certain extent if they provide this kind of empirical data it isn't clear why you should even *care* what their architecture is beyond intellectual curiosity.

Distributed systems solve a number of key problems at the heart of scaling large websites. I absolutely think this is going to become the default approach to handling state in internet application development. But no one benefits from the kind of irrational exuberance that currently surrounds the “big data” or nosql systems communities. This area is experiencing a boom—but nothing takes us from boom to bust like unrealistic hype and broken promises. We should be a little more honest about where these systems already shine and where they are still maturing.

Postscript: This blog seems to have resurfaced 7 years later. I actually still agree with much of it, though the dilemma is much less important. Distributed systems operations is hard, but you can now avoid it by just getting your distributed systems as a service. For example, [Confluent offers Kafka as a service with a contractual uptime SLA](#). Ops remains hard, but for distributed data systems you can make the people who wrote the software deal with the problem, and focus your time on your special sauce.

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · [SOURCE](#)

You can also read this post on [Prime Radiant's blog](#).

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

At [Prime Radiant](#), we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player Brainstorm app.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropdown folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills,

and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally
- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses superpowers-chrome to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The

arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called Slackline - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

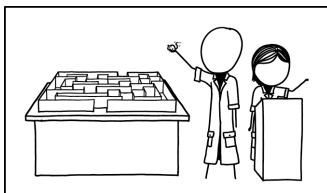
Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.

Maze

XKCD.COM · 03 AUG 2026 · [SOURCE](#)



RESEARCH BREAKTHROUGH: AFTER OVER A CENTURY OF TRIALS, SCIENTISTS HAVE FINALLY FOUND A MOUSE THAT CAN SOLVE THE MAZE.

As a side effect of the research, mice are now the only known animals other than humans to have developed a Backrooms mythology.